```
app.MapGet("/stock/{id?}", (int? id) => $"Received {id}");           ❶

app.MapGet("/stock2", (int? id) => $"Received {id}");                ❷

app.MapPost("/stock", (Product? product) => $"Received {product}");  ❸

app.Run();
```

❶ Uses a nullable simple type to indicate that the value is optional, so id is null when calling /stock

❷ This example binds to the query string. Id will be null for the request /stock2.

❸ A nullable complex type binds to the body if it's available; otherwise, it's null.

If no corresponding route value or query string contains the required value and the handler parameter is optional, the `EndpointHandler` uses `null` as the argument when invoking the endpoint handler. Similarly, for complex types that bind to the request body, if the request doesn't contain anything in the body and the parameter is optional, the handler will have a `null` argument.

WARNING If the request body contains the literal JSON value `null` and the handler parameter is marked optional, the handler argument will also be `null`. If the parameter isn't marked optional, you get the same error as though the request didn't have a body.

It's worth noting that you mark complex types binding to the request body as optional by using a *nullable reference type* (NRT) annotation: `?`. NRTs, introduced in C# 8, are an attempt to reduce the scourge of null-reference exceptions in C#, colloquially known as "the billion-dollar mistake." See http://mng.bz/vneM.

ASP.NET Core in .NET 7 is built with the assumption that NRTs are enabled for your project (and they're enabled by default in all the templates), so it's worth using them wherever you can. If you choose to disable NRTs